If a function uses `yield` instead of `return`, PHP creates an object with type `Generator` and interface `Iterator`. These objects behave like arrays (because of the `Iterator` interface) but only require enough memory to track the state of the currently yielded variable. For this reason, our peak memory usage is now around 1.2% of the previous example. It's a good idea to consider using generators for tasks like fetching lines from large files or iterating remote data sets. Generators allow you to break up intensive workloads into smaller jobs by only having to work on the subset of data that they yield on each iteration.

Summary

In this chapter, we have seen the fundamentals of PHP arrays and looked at many useful array-handling functions. You now know what arrays are, how to build them, and how to use them. Starting with simple indexed arrays, we moved on to associative arrays, then multi-dimensional array structures. We also examined techniques for iterating, combining, comparing, sorting, and searching.

It should be noted that most of the PHP array functions have both an associative and a recursive form. In this chapter, we've mainly focused on the simpler cases, but it's safe to assume that if an array function exists, there will be an associative and a recursive form of it. A good place to start if you want to learn more is the official PHP documentation for array functions: https://www.php.net/manual/en/ref.array.php

We also looked at functional ways to process array data, the techniques of mapping and reducing. The use of map/reduce functions is contentious; some developers swear by it while others swear *at* it. There is obviously a performance hit when using `array_reduce()` or `array_map()` vs. traditional loops because `for` and `foreach` do not introduce the overhead of placing function calls on the stack or forcing new arrays to be allocated. The tradeoff with map/reduce is that we can see at a glance that the code inside the mapping or reducer function will not have any side effects on the original data.